

# **An Unified Emergency Response Platform for Intelligent Ambulance Matching and Green Corridor Management**

## **PHASE I REPORT**

*Submitted by*

|                                |                  |
|--------------------------------|------------------|
| <b>KUMARAN S</b>               | <b>230701159</b> |
| <b>LOKESH SINGH N</b>          | <b>230701166</b> |
| <b>SUDHARSHAN KRISHNAA L K</b> | <b>230701350</b> |
| <b>VISHWAK S</b>               | <b>230701385</b> |

*in partial fulfillment for the award of the degree*

*of*

**BACHELOR OF ENGINEERING**

**IN**

**COMPUTER SCIENCE AND ENGINEERING**



**RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI**

**ANNA UNIVERSITY: CHENNAI 600 025**

**December 2026**

# **ANNA UNIVERSITY, CHENNAI**

## **BONAFIDE CERTIFICATE**

Certified that this Report titled “**AN UNIFIED EMERGENCY RESPONSE PLATFORM FOR INTELLIGENT AMBULANCE MATCHING AND GREEN CORRIDOR MANAGEMENT** ” is the bonafide work of **KUMARAN S (230701159), LOKESH SINGH N (230701166), SUDHARSHAN KRISHNAA L K (230701350), VISHWAK S (230701385)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

This project addresses the following Sustainable Development Goal(s):

SDG 3 : Good Health and Well-being

SDG 11 : Sustainable Cities and Communities

### **SIGNATURE**

Dr. J. Manoranjini  
Professor and Head  
Department of Computer Science  
and Engineering  
Rajalakshmi Engineering College  
Chennai – 602 105.

### **SIGNATURE**

Ms. Jaeyalakshmi M  
Assistant Professor (SS)  
Department of Computer Science  
and Engineering  
Rajalakshmi Engineering College  
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on \_\_\_\_\_

**Internal Examiner**

**External Examiner**

## **ABSTRACT**

India's rapidly growing urban population has placed increasing pressure on emergency healthcare infrastructure, with cities facing severe traffic congestion, uneven ambulance distribution, and fragmented communication between patients, ambulance providers, and hospitals. In such densely populated regions, even a well-intentioned emergency response system can fail simply because the right ambulance is not identified quickly enough, or because a hospital receives no advance warning of an incoming critical patient. These delays are especially costly within the Golden Hour, the critical early window during which timely medical intervention significantly improves a patient's chances of survival. UyirKappan is a unified emergency response platform built at the intersection of real-time distributed systems and geospatial computing. Rather than simply dispatching the nearest ambulance, the system intelligently evaluates multiple available ambulances based on real-time traffic conditions, distance, and availability, selecting a suitable ambulance to reach the patient quickly. If the chosen ambulance is unable to respond, the platform automatically finds and assigns the next best available option, ensuring the emergency request is not left unattended. Bystanders, ambulance drivers, and hospitals all remain connected through continuous live updates, allowing hospitals to prepare in advance for an incoming patient's arrival, including checking bed and resource availability. As a future direction, the platform also explores smart traffic signal coordination to give ambulances priority passage through busy intersections. By bringing together intelligent ambulance selection, reliable backup dispatch, and real-time coordination across all stakeholders, UyirKappan aims to reduce emergency response time and improve healthcare outcomes in fast-growing urban environments.

## **ACKNOWLEDGEMENT**

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **DR. J. MANORANJINI, M.E., Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **MS.JAEYALAKSHMI M, M.E.**, Assistant Professor (SS), Department of Computer Science and Engineering for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **DR. MADHUSUDHANAN, M.E., Ph.D.**, Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

**KUMARAN S : 230701159**

**LOKESH SINGH N : 230701166**

**SUDHARSHAN KRISHNAA L K : 230701350**

**VISHWAK S : 230701385**

## **TABLE OF CONTENTS**

| <b>CHAPTER NO.</b> | <b>TITLE</b>                  | <b>PAGE NO.</b> |
|--------------------|-------------------------------|-----------------|
|                    | <b>ABSTRACT</b>               | iii             |
|                    | <b>LIST OF TABLES</b>         | viii            |
|                    | <b>LIST OF FIGURES</b>        | ix              |
|                    | <b>LIST OF ABBREVIATIONS</b>  | x               |
| <b>1</b>           | <b>INTRODUCTION</b>           | <b>1</b>        |
|                    | 1.1 GENERAL                   | 1               |
|                    | 1.2 OBJECTIVE                 | 2               |
|                    | 1.3 EXISTING SYSTEM           | 2               |
|                    | 1.4 PROPOSED SYSTEM           | 3               |
| <b>2</b>           | <b>LITERATURE SURVEY</b>      | <b>6</b>        |
|                    | 2.1 LITERATURE REVIEW         | 6               |
|                    | 2.2 COMPARISON AND DISCUSSION | 11              |
|                    | 2.3 CONCLUSION                | 12              |
| <b>3</b>           | <b>SYSTEM DESIGN</b>          | <b>13</b>       |
|                    | 3.1 SYSTEM ARCHITECTURE       | 13              |
|                    | 3.2 SYSTEM REQUIREMENTS       | 15              |
|                    | 3.2.1 SOFTWARE REQUIREMENTS   | 15              |
|                    | 3.2.2 HARDWARE REQUIREMENTS   | 16              |

|   |                                                            |           |
|---|------------------------------------------------------------|-----------|
|   | 3.2.3 DATASET REQUIREMENTS                                 | 17        |
|   | 3.2.4 DEPLOYMENT AND SCALING                               | 17        |
|   | 3.3 SYSTEM DESIGN                                          | 18        |
|   | 3.3.1 ACTIVITY DIAGRAM                                     | 18        |
|   | 3.3.2 DATA FLOW DIAGRAM                                    | 19        |
| 4 | <b>PROJECT DESCRIPTION</b>                                 | <b>20</b> |
|   | 4.1 METHODOLOGIES                                          | 20        |
|   | 4.2 MODULES                                                | 21        |
|   | 4.2.1 BYSTANDER EMERGENCY<br>REQUEST APPLICATION           | 21        |
|   | 4.2.2 AMBULANCE DRIVER<br>APPLICATION                      | 22        |
|   | 4.2.3 HOSPITAL EMERGENCY<br>OPERATIONS DASHBOARD           | 22        |
|   | 4.2.4 CENTRAL REAL-TIME DISPATCH<br>BACKEND                | 23        |
|   | 4.2.5 INTELLIGENT MATCHER AND<br>ROUTE EVALUATION          | 23        |
|   | 4.2.6 CASCADING FALLBACK AND LIVE<br>TELEMETRY TRACKING    | 25        |
| 5 | <b>IMPLEMENTATION AND RESULT<br/>DISCUSSION</b>            | <b>26</b> |
|   | 5.1 IMPLEMENTATION                                         | 26        |
|   | 5.1.1 IMPLEMENTATION OF THE<br>EMERGENCY RESPONSE WORKFLOW | 26        |
|   | 5.1.2 IMPLEMENTATION OF REAL-TIME<br>TRACKING AND ETA      | 27        |

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
|          | 5.1.3 IMPLEMENTATION OF<br>CASCADING FALLBACK AND HOSPITAL<br>COORDINATION | 28        |
|          | 5.2 RESULTS AND DISCUSSION                                                 | 31        |
| <b>6</b> | <b>CONCLUSION AND FUTURE WORK</b>                                          | <b>33</b> |
|          | 6.1 CONCLUSION                                                             | 33        |
|          | 6.2 FUTURE WORK                                                            | 33        |
|          | <b>REFERENCES</b>                                                          | <b>42</b> |

## **LIST OF TABLES**

| <b>TABLE NO</b> | <b>TABLE NAME</b>                            | <b>PAGE NO</b> |
|-----------------|----------------------------------------------|----------------|
| 3.2.1           | Software requirements of the proposed system | 15             |
| 3.2.2           | Hardware requirements of the proposed system | 16             |

## LIST OF FIGURES

| <b>FIGURE NO.</b> | <b>FIGURE NAME</b>                                                | <b>PAGE NO.</b> |
|-------------------|-------------------------------------------------------------------|-----------------|
| 1.1               | Overview of the Proposed System Architecture                      | 4               |
| 3.1               | System Architecture of the Proposed Uyirkappan Platform           | 13              |
| 3.2               | Activity Diagram                                                  | 18              |
| 3.3               | Data Flow Diagram of the System                                   | 19              |
| 5.1               | Bystander Application : Emergency Request Interface               | 27              |
| 5.2               | Bystander Application : Live Tracking with ETA and Route          | 28              |
| 5.3               | Driver Application : New Emergency Assignment with Response Timer | 29              |
| 5.4               | Hospital Dashboard : Emergency Command and Inbound Patient Alert  | 30              |
| 5.5               | Hospital Dashboard : Response History and Metrics                 | 30              |
| 5.6               | Mean Dispatch ETA Comparison                                      | 31              |
| 5.7               | Mean ETA Comparison under Different Traffic Conditions            | 32              |

## **LIST OF ABBREVIATIONS**

**API** – Application Programming Interface

**CPR** – Cardiopulmonary Resuscitation

**CPU** – Central Processing Unit

**EMS** – Emergency Medical Services

**ETA** – Estimated Time of Arrival

**GPS** – Global Positioning System

**H3** – Hierarchical Hexagonal Geospatial Indexing System

**ICU** – Intensive Care Unit

**JWT** – JSON Web Token

**LTS** – Long-Term Support

**RAM** – Random Access Memory

**REST** – Representational State Transfer

**SDG** – Sustainable Development Goal

**ANN** – Artificial Neural Network

**FDNY** – Fire Department of the City of New York

**GIS** – Geographic Information System

**MDP** – Markov Decision Process

# **CHAPTER 1**

## **INTRODUCTION**

### **1.1 GENERAL**

Rapid urbanization, population growth, and vehicular congestion in developing metropolitan regions have created unprecedented pressure on emergency medical transport systems. In acute clinical scenarios such as road traffic trauma, acute myocardial infarction (STEMI), ischemic stroke, and severe respiratory failure, the patient's survival probability is governed by the "Golden Hour" principle. Clinical research indicates that pre-hospital intervention within the initial 60 minutes following trauma reduces mortality rates by up to 63%.

Despite advances in medical science, emergency transport logistics remain a weak link in urban healthcare infrastructure. When emergency dispatch relies on antiquated manual workflows, vital pre-hospital time is lost before an ambulance arrives at the scene. Furthermore, pre-hospital transit is treated as an isolated event, with receiving hospitals having little to no advance information regarding the inbound patient's vital signs, expected arrival time, or specialized resource requirements.

UyirKappan (meaning "Life Savior") was designed to modernize emergency logistics. It unites the bystander, the ambulance driver, the dispatch center, and the receiving hospital into an automated, synchronized, real-time ecosystem powered by traffic-weighted graph algorithms and WebSocket event streams.

The platform focuses on reducing delays in ambulance selection, assignment, and coordination by maintaining a continuous flow of information among all stakeholders. Instead of depending only on the geographically nearest ambulance, it evaluates available candidates based on route conditions and availability. A cascading fallback mechanism further ensures that an emergency request can be reassigned when an ambulance rejects or fails to respond. This approach aims to provide a more reliable and coordinated emergency response workflow.

## **1.2 OBJECTIVE**

The primary objective of UyirKappan is to reduce emergency dispatch delays, improve ambulance selection, provide automatic reassignment during failed dispatch attempts, and coordinate hospitals through an integrated digital platform. The system aims to provide a simple interface for bystanders to submit emergency information and location, while using geospatial and route-based computation to identify a suitable ambulance. It further aims to maintain continuous communication among bystanders, ambulance providers, and hospitals throughout the emergency-response process.

The specific objectives are to develop a single-tap emergency reporting interface, perform ambulance selection using H3-based spatial filtering and Dijkstra-based route-cost evaluation, and implement a cascading fallback mechanism that automatically reassigns requests when an ambulance rejects or fails to respond. The platform also aims to provide real-time ambulance location, status, and estimated time of arrival updates, while enabling hospitals to view incoming emergency cases and maintain resource-availability information. These objectives establish a coordinated workflow from emergency reporting and ambulance assignment to hospital preparation and arrival.

## **1.3 EXISTING SYSTEM**

Existing emergency medical response systems commonly rely on centralized call-center models in which emergency incidents are received through a public-safety answering point and then manually verified and dispatched to an available ambulance. This workflow introduces delays during incident intake, verification, and communication between the caller and ambulance provider. The process may also depend on verbal descriptions of locations or landmarks, which can result in inaccurate or delayed ambulance routing.

Conventional digital ambulance dispatch systems often select vehicles based primarily on geographic distance. Static distance measures such as Haversine or Euclidean distance do not always represent actual road travel conditions, particularly in dense urban environments with

one-way roads, bottlenecks, and congestion. As a result, the geographically nearest ambulance may not always be the ambulance capable of reaching the emergency location fastest.

Some existing approaches adapt ride-hailing or nearest-neighbor dispatch techniques for emergency transportation. Although these approaches can simplify vehicle selection, they may not adequately consider medical requirements, ambulance capabilities, or coordination with receiving hospitals. Periodic client polling can also delay the exchange of ambulance status and location information, limiting continuous visibility during an active emergency.

Other research focuses on dynamic ambulance relocation and multi-agent dispatch, where ambulance fleets are positioned according to historical emergency demand. These methods can improve the availability of ambulances in high-demand areas, but centralized optimization approaches may introduce computational overhead and are primarily designed for fleet positioning rather than immediate roadside dispatch. Existing approaches often address individual aspects of emergency response rather than providing a unified workflow connecting bystanders, ambulance providers, and hospitals. UyirKappan addresses this gap by integrating emergency reporting, ambulance matching, cascading reassignment, live tracking, and hospital coordination.

## **1.4 PROPOSED SYSTEM**

### **A. System Overview**

UyirKappan is a unified emergency response platform designed to coordinate bystanders, ambulance drivers, and hospitals through a centralized digital system. The platform begins when a bystander submits an emergency request containing the required emergency details and location. The Backend API receives and manages the request, while the matching process identifies suitable ambulances based on their availability and location. Once an ambulance is assigned, the system maintains the emergency request as a continuous workflow rather than treating dispatch as a single transaction. This enables the different participants to remain connected throughout the emergency-response process.

## B. System Architecture

The proposed architecture consists of three client-facing components: the Bystander Application, Driver Application, and Hospital Dashboard, connected through a centralized Backend API/Server. The backend coordinates request management, ambulance location updates, notifications, matching, fallback dispatch, and database operations. The Matching Engine identifies suitable ambulance candidates using H3-indexed geospatial matching, while the Cascading Fallback Dispatch component handles rejected or unanswered assignments. The architecture also includes a real-time communication channel for distributing ambulance status and location updates, together with a database for persistent emergency and operational information. The overall interaction among these components is illustrated in Fig. 1.

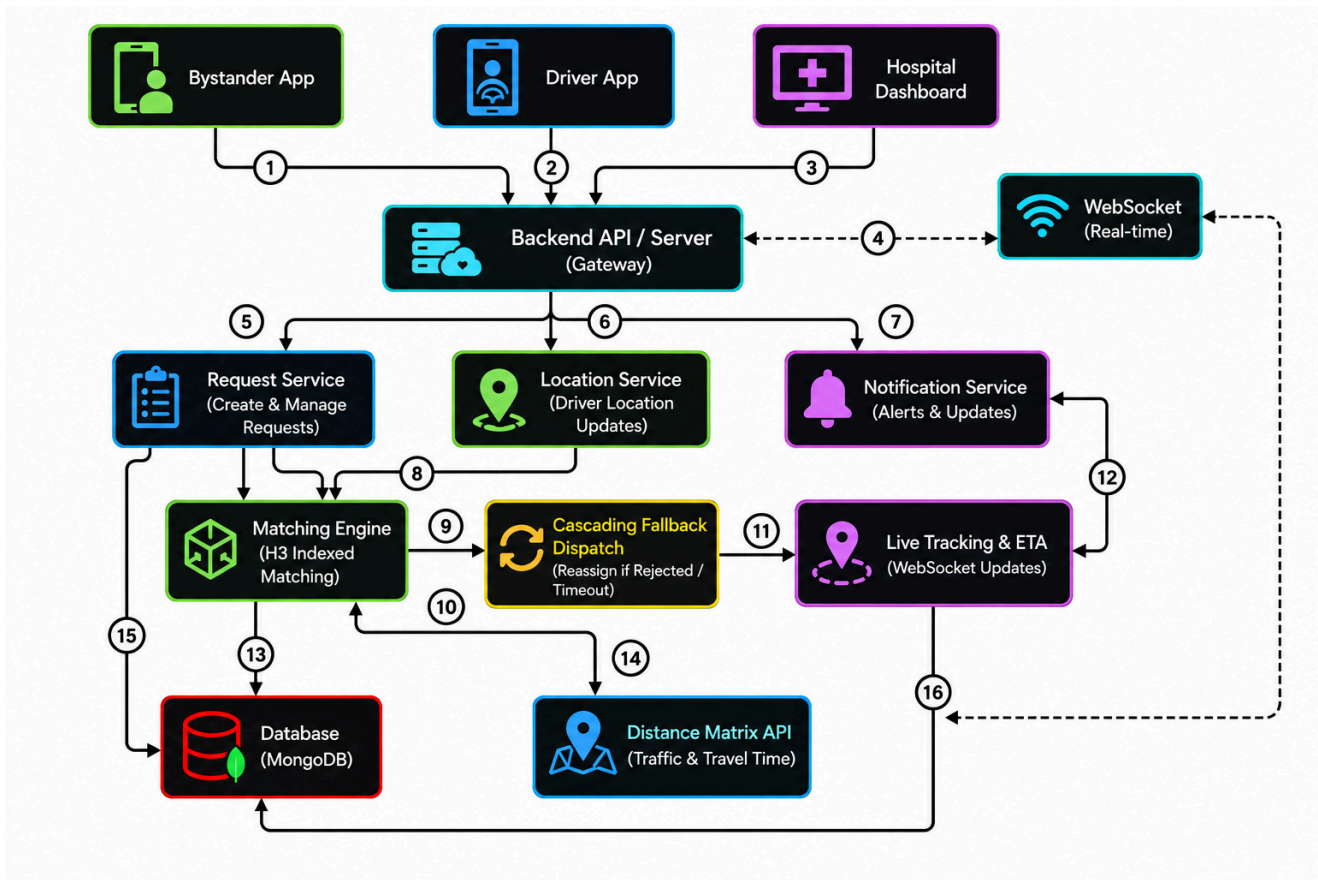


Figure. 1.1 Overview of the Proposed System architecture

### **C. Emergency Request and Ambulance Matching**

The Bystander Application provides the interface for creating an emergency request by capturing the emergency category, victim information, and geographical location. After the request reaches the backend, the Matching Engine identifies available ambulance candidates using H3-based spatial filtering and evaluates the relevant candidates using route-based computation. This approach reduces dependence on simple geographic proximity by considering the available road network during ambulance selection. The selected ambulance receives the assignment through the Driver Application and can respond to the request. The system records the assignment outcome so that the emergency request can proceed through the appropriate response state.

### **D. Cascading Fallback and Hospital Coordination**

When an assigned ambulance rejects the request or fails to respond within the configured response period, the Cascading Fallback Dispatch mechanism excludes that ambulance from the current request and attempts reassignment to the next suitable candidate. Once an ambulance accepts the request, its location and status can be communicated through the real-time channel to the Bystander Application and Hospital Dashboard. The hospital interface provides information about incoming emergency cases and allows hospital staff to maintain resource-availability information. Thus, the proposed system integrates emergency reporting, ambulance assignment, automatic fallback, live tracking, and hospital coordination into a single continuous emergency-response workflow.

## **CHAPTER 2**

### **LITERATURE SURVEY**

This section of the paper, provides an all-round review about multiple existing systems that hold similar objective and problem statements as that of the proposed project. The study of these existing papers will produce valuable insights for the construction of the proposed system.

#### **2.1 LITERATURE REVIEW**

Rafaqat et al. [1] presented EMCON, a digital emergency-response platform designed to connect patients, ambulances, and hospitals through a common platform. The system supports real-time resource availability, ambulance coordination, inter-hospital transfers, and communication among emergency stakeholders. The study demonstrates the importance of integrating different participants involved in emergency response instead of treating ambulance and hospital services independently. However, the system does not focus on intelligent ambulance matching based on multiple factors and does not provide traffic-aware route optimization or a cascading mechanism for handling rejected or unanswered ambulance assignments.

Smith et al. [2] evaluated the GoodSAM volunteer first-responder application, which is integrated with ambulance computer-aided dispatch systems. The application uses smartphones and GPS to automatically alert nearby CPR-trained volunteers when an out-of-hospital cardiac arrest occurs, allowing intervention before the arrival of emergency medical services. The study demonstrates how mobile applications and location-based services can support faster emergency assistance by involving nearby trained responders. However, the system is primarily focused on cardiac-arrest incidents and volunteer response, rather than general ambulance dispatch and coordination among bystanders, ambulance drivers, and hospitals.

Yang et al. [3] proposed a personalized dispatch-menu optimization approach for on-demand mobility services. The approach uses probabilistic modelling to estimate driver acceptance and

rejection behaviour and uses these predictions to improve dispatch decisions. The study reports that the proposed approach can reduce unmatched orders and improve driver utilization. Although the approach is relevant to assignment problems involving uncertain driver acceptance, it is designed for ride-hailing applications and does not consider emergency-specific factors such as patient severity, ambulance capabilities, hospital suitability, or emergency response time.

Fu et al. [4] proposed a coupled Game-Theoretical and Discrete-Event Simulation framework for modelling ambulance dispatch and hospital decision-making. The framework considers factors including hospital capacity, travel time, and queuing behaviour and models the interaction between ambulance services and hospitals. The study demonstrates how both ambulance movement and hospital capacity can influence emergency-response decisions. However, the model relies on simulation and simplifying assumptions and mainly represents hospital behaviour through predefined decision strategies. It does not provide a complete real-time operational platform for ambulance assignment and emergency coordination.

Olivier et al. [5] developed a probabilistic hospital recommendation system using historical FDNY EMS data and vehicle telematics. The system considers uncertainty in travel-time estimation while recommending suitable hospitals for emergency cases. The study highlights the importance of incorporating travel-time uncertainty when selecting a receiving hospital rather than considering hospital distance alone. However, the approach primarily focuses on hospital recommendation and does not address the complete emergency-response process involving ambulance selection, dispatch, real-time tracking, and coordination between multiple stakeholders.

Xu et al. [6] proposed a simulation-based EMS hospital recommendation system using Discrete Event Simulation and Google Maps. The approach predicts future emergency department and intensive care unit bed availability over different time intervals, including 20, 40, and 60 minutes, to support hospital selection. The study demonstrates the importance of considering

future hospital resource availability when recommending a destination for an emergency case. However, the system was evaluated using historical and simulated data rather than real-time operational conditions, and prediction performance varies between hospitals. It also focuses mainly on hospital recommendation rather than complete ambulance coordination.

Zaki et al. [7] introduced a cloud-assisted microservice-based healthcare application framework using cloud deployment and DevOps practices. The proposed architecture focuses on scalability, loose coupling, interoperability, and independent deployment of healthcare services. The study shows how microservice-based architectures can support the development of scalable healthcare applications by separating different services. However, the framework does not propose a specific ambulance matching, routing, or optimization algorithm. Therefore, while the architectural approach can support healthcare platforms, additional decision-making mechanisms are required for emergency ambulance coordination.

Ankarboina et al. [8] proposed RACER, a real-time congestion-aware emergency-routing framework designed for emergency vehicles. The framework uses multi-edge look-ahead and a congestion-aware version of Dijkstra's algorithm to identify suitable routes while reducing unnecessary rerouting and communication overhead. The study demonstrates the importance of incorporating real-time congestion information into emergency vehicle routing. However, the framework depends on real-time edge-level traffic information obtained through mechanisms such as vehicle-to-infrastructure communication or telemetry and primarily addresses routing for a single emergency vehicle rather than multi-ambulance resource allocation and assignment.

Xu and Li [9] proposed a Dynamic Risk Field Model for predicting urban traffic congestion in real time. The model combines traffic information, historical traffic patterns, road topology, incidents, and weather conditions to construct a spatial congestion-risk field. The approach provides a method for representing congestion as a spatially varying risk rather than relying only on static traffic information. However, its performance depends on the availability and quality of traffic sensor or probe data, while Gaussian-kernel parameters require calibration.

Complex traffic incidents are also simplified through Gaussian decay, which can limit the representation of certain real-world traffic conditions.

Nozari et al. [10] developed an ambulance-routing optimization model that combines neural-network-based learning with a Genetic Algorithm. The approach aims to reduce ambulance response time and transportation costs while increasing the number of emergency cases that can be handled. The study demonstrates how machine-learning techniques can be combined with evolutionary optimization for ambulance-routing decisions. However, the model considers a limited set of variables and does not fully incorporate real-time factors such as traffic congestion, ambulance availability, hospital capacity, and road conditions within the stated dataset.

Abdeen et al. [11] proposed a smart ambulance model that considers ambulance response, road congestion, and hospital loading during both the ambulance-to-patient and patient-to-hospital stages. The approach also considers hospital selection based on the minimum expected door-to-needle time. The study demonstrates that emergency response can benefit from considering both ambulance travel and hospital conditions rather than optimizing only the first stage of the journey. However, the approach is primarily a simulation-based decision model and does not provide a complete multi-stakeholder platform. It also does not address request-level cascading fallback when an assigned ambulance rejects an assignment or fails to respond within a specified time.

Schmid [12] proposed an Approximate Dynamic Programming approach for dynamic ambulance relocation and dispatching under stochastic conditions. The method jointly considers ambulance dispatch and relocation decisions and aims to reduce average ambulance response time under changing emergency conditions. The study demonstrates the potential of dynamic optimization for improving ambulance deployment decisions. However, Approximate Dynamic Programming faces the curse of dimensionality and requires state aggregation, creating a trade-off between computational complexity and solution accuracy. The approach also does not

fully integrate real-time traffic information, machine-learning-based prediction, or continuous coordination among multiple emergency stakeholders.

Tluli et al. [13] proposed a dynamic redeployment system for Critical Care Response Vehicles using Deep Reinforcement Learning. The problem is modelled as a Markov Decision Process, with a Deep Scoring Network and the REINFORCE policy-gradient method used to determine redeployment decisions. The approach focuses on proactively positioning emergency vehicles based on changing operational conditions. The study demonstrates the potential of reinforcement learning for dynamic emergency-vehicle redeployment. However, the approach requires historical and operational data for training and focuses mainly on proactive vehicle redeployment rather than request-level ambulance assignment and cascading fallback after an ambulance rejects or fails to respond to an emergency request.

Chu et al. [14] proposed a Deep Encoder Cross Network for estimated time of arrival prediction. The model uses traffic, weather, road-speed, congestion, and other proxy features to improve ETA estimation and particularly addresses cases involving late arrivals. The study demonstrates the importance of combining multiple environmental and traffic-related features for accurate travel-time prediction. However, the proposed deep-learning architecture requires substantial training data and has relatively high model complexity. Its prediction performance also depends on the quality and availability of the traffic, weather, and other proxy features used by the model.

Abid et al. [15] developed an ambulance travel-time estimation framework using spatiotemporal ambulance data collected from SOS Alarm in Skåne, Sweden. The study compares Polynomial Regression, XGBoost, and Artificial Neural Network models for ambulance travel-time estimation, with XGBoost and ANN providing better estimation performance than Polynomial Regression. The work demonstrates the usefulness of historical ambulance data and machine-learning techniques for estimating emergency travel time. However, the approach

relies mainly on historical ambulance data rather than fully real-time traffic information, while Google Maps travel time introduces a dependency on an external routing estimate. Weather and other real-time environmental factors are also not incorporated.

## **2.2 COMPARISON AND DISCUSSION**

From the literature reviewed in Section 2.1, it can be observed that existing emergency-response systems address different aspects of ambulance and hospital coordination. Rafaqat et al. [1] and Smith et al. [2] focus on communication and coordination between emergency stakeholders, while Yang et al. [3] and Fu et al. [4] investigate dispatch and decision-making approaches. Hospital selection and resource availability are considered by Olivier et al. [5] and Xu et al. [6]. These studies show that effective emergency response requires coordination between multiple stakeholders rather than relying only on ambulance movement.

The reviewed studies also show the importance of considering traffic and route conditions during emergency response. Ankarboina et al. [8] use congestion-aware Dijkstra for emergency routing, while Xu and Li [9] investigate traffic-congestion prediction using multiple spatial and temporal factors. Nozari et al. [10] combine neural-network-based learning with Genetic Algorithm for ambulance routing, and Abdeen et al. [11] consider ambulance response, road congestion, and hospital loading together. These approaches demonstrate that selecting an ambulance or route only according to geographical distance may not always provide an appropriate emergency-response decision.

Several studies investigate optimization and learning-based approaches for ambulance allocation and travel-time estimation. Schmid [12] applies Approximate Dynamic Programming for ambulance relocation and dispatching, while Tluli et al. [13] use Deep Reinforcement Learning for dynamic redeployment of critical care response vehicles. Chu et al. [14] and Abid et al. [15] focus on ETA and ambulance travel-time estimation using deep-learning and machine-learning approaches. Although these approaches improve individual decision-making tasks, they generally require historical or operational data and focus on a specific part of the

emergency-response process.

Based on the comparison, there is a need for an integrated emergency-response platform that combines geospatial ambulance filtering, route-based ambulance selection, real-time ambulance tracking, ETA updates, fallback dispatch, and hospital coordination. The proposed system addresses these requirements by connecting the Bystander Application, Driver/Ambulance Application, and Hospital Dashboard through a common backend. H3-based spatial filtering is used to identify relevant ambulance candidates, while Dijkstra-based route computation supports route-cost evaluation. The system also provides cascading fallback when an ambulance rejects or fails to respond to an assignment and maintains real-time communication between the connected applications.

## **2.3 CONCLUSION**

The literature survey shows that existing studies have addressed several important aspects of emergency response, including communication, ambulance dispatch, hospital recommendation, traffic-aware routing, ambulance allocation, and ETA estimation. These studies demonstrate the importance of considering factors such as travel time, traffic conditions, ambulance availability, and hospital resources. However, most existing approaches focus on individual components rather than providing a unified emergency-response workflow. Automatic fallback after ambulance rejection or timeout and real-time coordination among bystanders, ambulance services, and hospitals are not consistently addressed together.

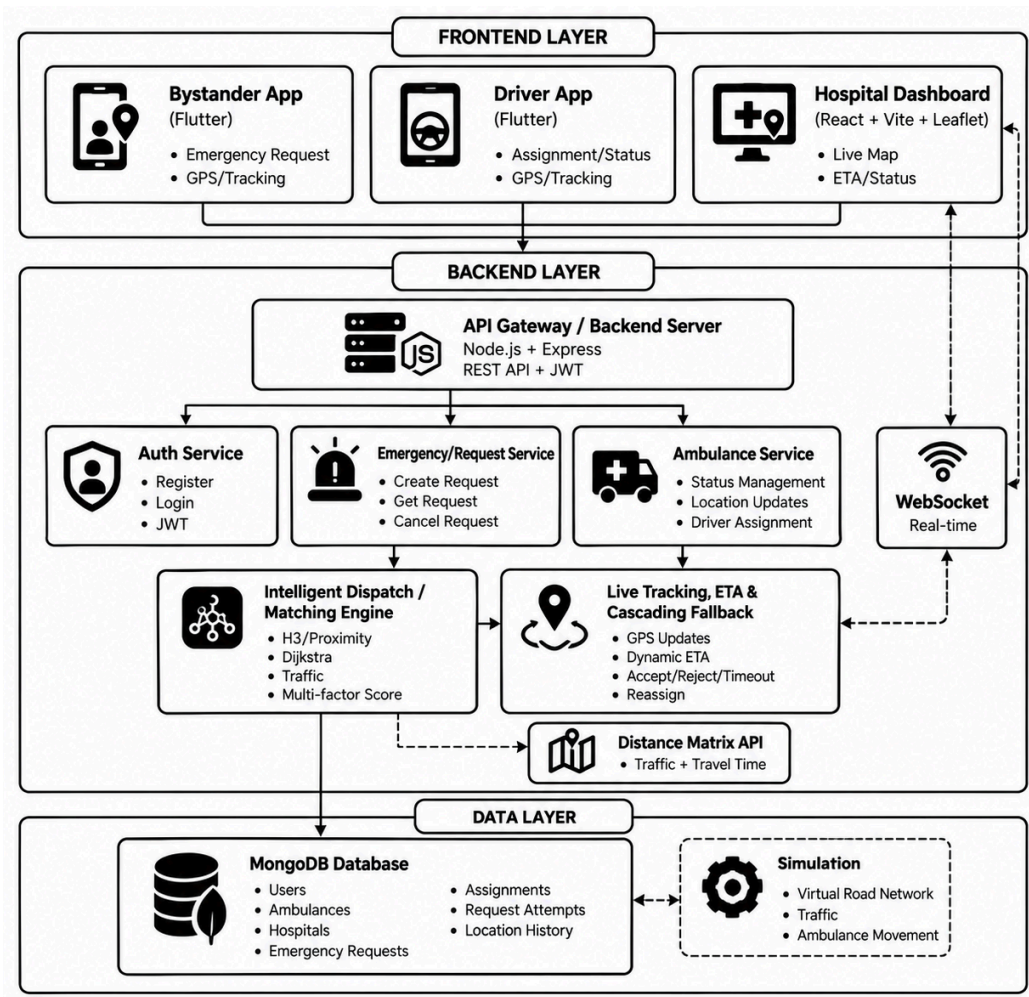
Therefore, the proposed system integrates ambulance candidate filtering, route evaluation, multi-factor selection, cascading fallback dispatch, real-time tracking, ETA updates, and hospital resource visibility through a shared backend. This provides a coordinated workflow from emergency request initiation to hospital arrival.

## CHAPTER 3

### SYSTEM DESIGN

This chapter briefly discusses the design of the proposed system with the help of Model diagrams. The requirements of the system and deployment are also extensively studied. This chapter also covers the overall architecture of the proposed system along with the overview of modules involved.

#### 3.1 SYSTEM ARCHITECTURE



**Figure. 3. 1. System architecture of the proposed UyirKappan platform.**

The proposed system follows a layered architecture consisting of the Frontend Layer, Backend Layer, and Data Layer. The architecture connects the Bystander App, Driver App, and Hospital Dashboard through a centralized backend. The Bystander App and Driver App are developed

using Flutter, while the Hospital Dashboard uses React, Vite, and Leaflet for web-based visualization. The Bystander App is responsible for submitting emergency requests and sharing location information, whereas the Driver App handles ambulance assignments, status updates, and GPS tracking. The Hospital Dashboard provides hospital personnel with live map information and ambulance ETA and status updates.

The Backend Layer acts as the central processing and communication layer of the platform. The API Gateway/Backend Server is implemented using Node.js and Express and provides REST APIs with JWT-based authentication. Separate services manage authentication, emergency requests, and ambulance operations. The Emergency/Request Service handles the creation, retrieval, and cancellation of emergency requests, while the Ambulance Service manages ambulance status, location updates, and driver assignments. The Intelligent Dispatch/Matching Engine uses H3-based proximity filtering, Dijkstra-based route computation, traffic information, and a multi-factor score to identify suitable ambulances. A Distance Matrix API is used to obtain traffic and travel-time information for route evaluation.

The backend also manages real-time tracking, ETA calculation, and cascading fallback dispatch. Ambulance GPS updates are received through the ambulance service and used to update the current location and estimated time of arrival. WebSocket-based communication enables real-time updates to the connected applications, allowing the Bystander App and Hospital Dashboard to receive changes without repeatedly requesting the server. When an assigned driver accepts, rejects, or fails to respond within the configured time, the assignment status is recorded and the fallback mechanism can initiate reassignment to another eligible ambulance. This allows ambulance assignment, location, ETA, and status information to remain synchronized across the platform.

The Data Layer uses MongoDB to store the major operational information required by the platform. The database maintains user information, ambulance details, hospital information, emergency requests, assignments, request attempts, and ambulance location history. A

simulation component is also included for testing the emergency workflow using a virtual road network, traffic conditions, and simulated ambulance movement. This layered architecture separates user interfaces, backend processing, real-time communication, and data management, allowing the different components of the emergency-response workflow to operate together through a common system.

## 3.2 SYSTEM REQUIREMENTS

The system requirements define the software, hardware, data, and deployment resources required to develop and operate the UyirKappan platform. The requirements cover the mobile applications used by bystanders and ambulance drivers, the web-based hospital interface, the backend services, the database, and the routing and real-time communication components.

### 3.2.1 SOFTWARE REQUIREMENTS

The software requirements include the programming languages, frameworks, runtime environments, database technologies, and supporting tools required to implement the different components of the system.

| Component     | Version | Purpose                                  |
|---------------|---------|------------------------------------------|
| Flutter       | 3.29.2  | Bystander and Driver mobile applications |
| Dart          | 3.x     | Mobile application development           |
| Node.js       | 22 LTS  | Backend runtime                          |
| Express.js    | 5.x     | REST API development                     |
| MongoDB       | 8.x     | Emergency and system data storage        |
| Socket.IO     | 4.x     | Real-time communication                  |
| React         | 18.x    | Hospital dashboard interface             |
| JavaScript    | ES2022  | Backend and dashboard                    |
| OpenStreetMap | —       | Map and location visualization           |

|                   |       |                                          |
|-------------------|-------|------------------------------------------|
| H3                | —     | Spatial indexing and candidate filtering |
| Swagger / OpenAPI | 3.0.3 | API documentation and testing            |

**Table 3.1: Software Requirements of Proposed System**

### 3.2.2 HARDWARE REQUIREMENTS

The hardware requirements specify the basic computing, memory, storage, network, and location capabilities required by the backend infrastructure and client devices. The server requires sufficient processing and memory resources for handling concurrent emergency requests, database operations, matching computations, and real-time communication, while mobile devices require GPS and mobile-network connectivity for ambulance tracking.

| Component            | Requirement               | Purpose                                              |
|----------------------|---------------------------|------------------------------------------------------|
| Backend Server       | 2+ CPU cores, 4 GB RAM    | API processing, matching and real-time communication |
| Database Server      | 2+ CPU cores, 4 GB RAM    | Storage and retrieval of emergency data              |
| Smartphone           | Android/iOS with GPS      | Bystander and driver applications                    |
| Network Connectivity | Wi-Fi/4G/5G               | Real-time communication and location updates         |
| GPS Module           | Smartphone-integrated GPS | Ambulance and emergency location tracking            |

**Table 3.2: Hardware Requirements of Proposed System**

### **3.2.3 DATASET REQUIRED**

The system requires structured data representing emergency requests, ambulance availability, geographical locations, road-network information, and hospital resources. Emergency-request data include the emergency category, victim count, requester location, and relevant timestamps. Ambulance data include vehicle identification, availability status, geographical position, and H3 cell information. The routing component requires a road-network graph containing nodes, directed edges, distances, and travel-time information for route computation. Hospital data include hospital identification and available resources such as general beds, intensive care beds, and ventilator availability. For controlled evaluation, simulated emergency requests and ambulance movements can be generated from these data categories.

The current system specification defines a virtual road-network graph using nodes and directed edges with corresponding distances and nominal travel times. This graph provides the routing data required for evaluating ambulance routes using the proposed routing method.

### **3.2.4 DEPLOYMENT AND SCALING**

The proposed system is designed as a modular client-server architecture in which the mobile applications and Hospital Dashboard communicate with centralized backend services. The backend separates request processing, location management, notification handling, ambulance matching, fallback dispatch, and data persistence into distinct functional components. This separation allows individual services to be maintained or scaled according to their workload. As the number of emergency requests and connected clients increases, additional backend instances and database resources can be introduced while maintaining the same client interfaces and communication workflow. Real-time communication channels can similarly be managed independently to support increasing numbers of connected users.

### 3.3 SYSTEM DESIGN

The system design describes the functional behaviour and data movement of the UyirKappan platform. It represents how emergency information moves between the user applications, backend services, matching components, database, and hospital interface during an emergency-response workflow.

#### 3.3.1 ACTIVITY DIAGRAM

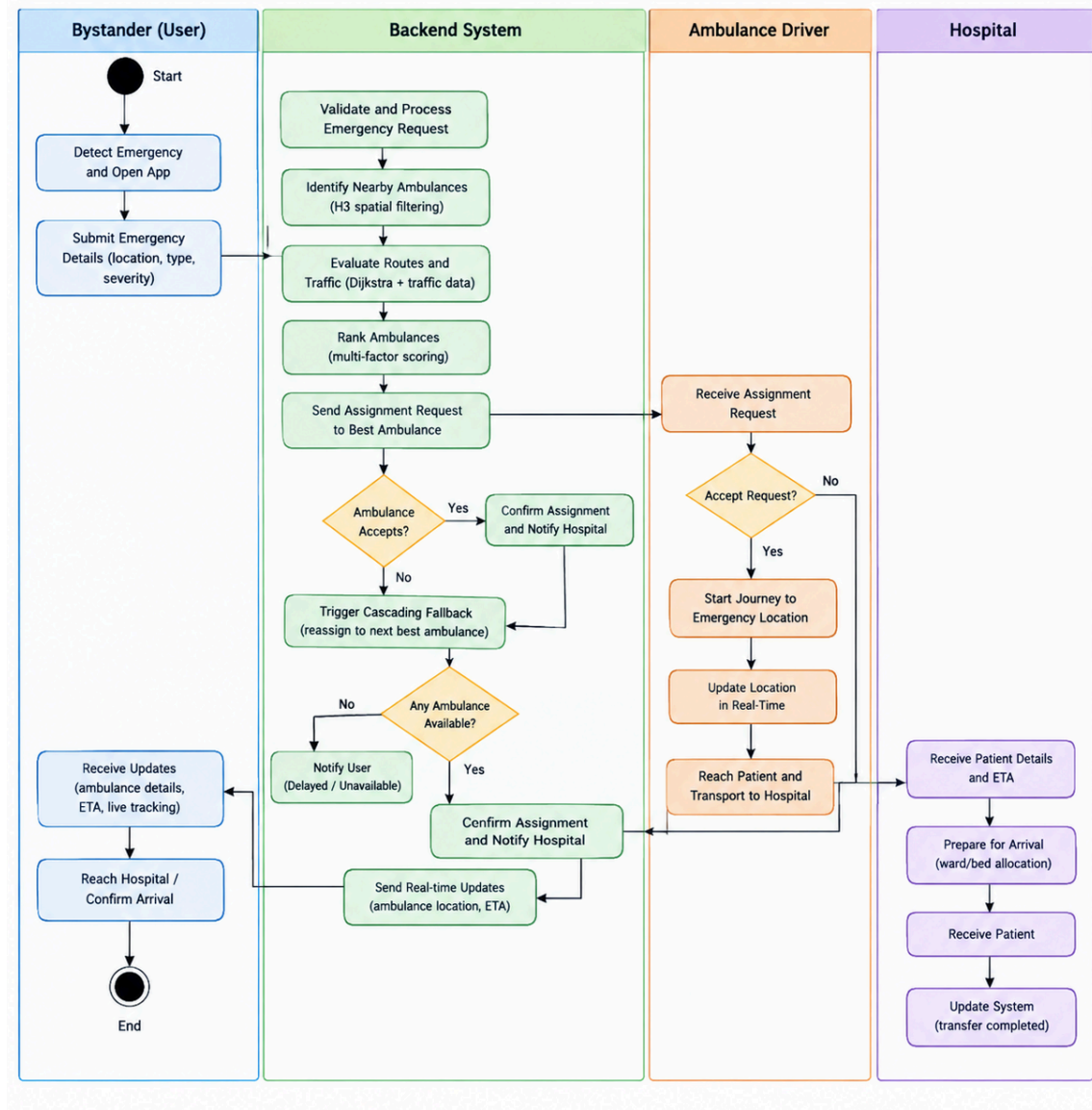


Figure. 3.2 Activity Diagram

### 3.3.2 DATA FLOW DIAGRAM

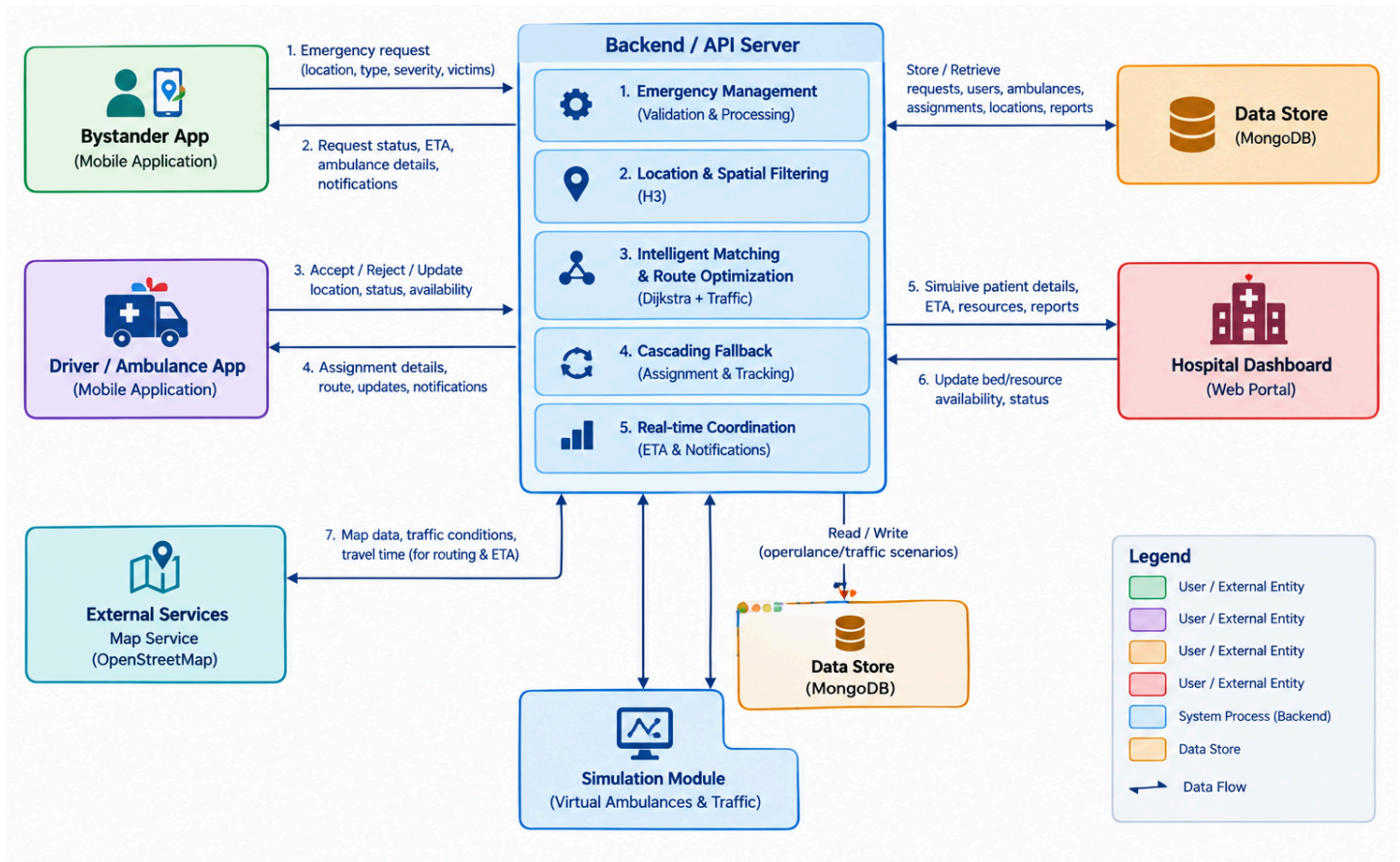


Figure. 3.3 Data Flow Diagram of the System

## **CHAPTER 4**

### **PROJECT DESCRIPTION**

#### **4.1 METHODOLOGY**

The proposed UyirKappan platform follows an integrated emergency-response methodology that connects the bystander, ambulance driver, and hospital through a centralized backend. The process begins when a bystander submits an emergency request containing the emergency category, victim count, and geographical location. The request is validated and stored by the backend, after which the ambulance matching process identifies suitable available vehicles. This approach treats emergency handling as a continuous workflow covering request creation, ambulance assignment, response, tracking, and hospital coordination.

The ambulance selection process combines geospatial filtering with route-based evaluation. Ambulance locations are represented using H3 spatial indexes to identify vehicles within the relevant search area. The resulting candidates are evaluated using Dijkstra-based route computation over the available road-network graph. Candidate factors such as travel time, physical distance, traffic-related cost, and ambulance availability can then be considered for ranking. The highest-ranked eligible ambulance is offered the emergency assignment through the Driver Application.

To improve reliability, the system uses a cascading fallback mechanism for unsuccessful assignments. When an ambulance rejects a request or fails to respond within the configured response period, the assignment is recorded and the ambulance is excluded from subsequent attempts for the same request. The matching process is then repeated using the remaining eligible candidates. Once an ambulance accepts the request, its operational status and location are continuously updated, allowing the Bystander Application and Hospital Dashboard to receive the latest emergency information.

The final stage involves real-time coordination and hospital preparedness. Ambulance location and status updates are transmitted through the real-time communication layer, while emergency

information and estimated time of arrival are made available to the receiving hospital. Hospital staff can maintain resource-availability information through the dashboard, allowing the system to provide a common operational view of the emergency. Thus, the methodology integrates emergency reporting, intelligent ambulance matching, automated fallback, live tracking, and hospital coordination into a single response workflow.

## **4.2 MODULES**

The UyirKappan platform is divided into six major modules, with each module responsible for a specific stage of the emergency-response workflow. The modules collectively provide emergency request management, ambulance-side interaction, hospital coordination, backend processing, intelligent matching, and fault-tolerant dispatch with real-time tracking.

### **4.2.1 MODULE 1: BYSTANDER EMERGENCY REQUEST APPLICATION**

The Bystander Emergency Request Application provides the primary interface for reporting an emergency. It allows the user to select the emergency category, specify the number of victims, and provide the emergency location using the device's location services. The request is transmitted to the backend for validation and processing. After an ambulance is assigned, the application provides the user with relevant information about the assigned vehicle and the progress of the emergency request. The application therefore simplifies emergency reporting while providing continuous visibility of the active response.

Key functions:

- Emergency request creation
- GPS-based location capture
- Emergency category selection
- Victim-count specification
- Ambulance assignment status
- Live ambulance location and estimated time of arrival updates

#### **4.2.2 MODULE 2: AMBULANCE DRIVER APPLICATION**

The Ambulance Driver Application enables ambulance providers to receive and respond to emergency assignments. When a suitable ambulance is selected, the driver receives the emergency request along with relevant pickup and routing information. The driver can accept or reject the assignment within the configured response period. After acceptance, the application supports the progression of the ambulance through the defined emergency-response stages and provides location information to the backend for tracking.

Key functions:

- Ambulance availability management
- Emergency assignment notification
- Assignment acceptance and rejection
- Response-stage updates
- GPS location transmission
- Ambulance status updates

#### **4.2.3 MODULE 3: HOSPITAL EMERGENCY OPERATIONS DASHBOARD**

The Hospital Emergency Operations Dashboard provides hospital staff with visibility into incoming emergency cases and available hospital resources. Once an ambulance is assigned to a hospital, relevant emergency information and estimated time of arrival can be displayed through the dashboard. Hospital staff can also update supported resource categories such as general beds, intensive care beds, and ventilator availability. The dashboard provides a centralized interface for monitoring incoming emergencies and maintaining current resource information.

Key functions:

- Incoming emergency case monitoring
- Ambulance and estimated time of arrival visibility

- Emergency category and victim information
- Hospital resource-availability management
- Assignment and fallback status visibility
- Emergency case status monitoring

#### **4.2.4 MODULE 4: CENTRAL REAL-TIME DISPATCH BACKEND**

The Central Real-Time Dispatch Backend acts as the coordination layer between the client applications and the system's data services. It receives emergency requests, manages ambulance assignments, processes driver responses, maintains emergency states, and distributes relevant updates to connected clients. REST-based services support request and data operations, while real-time communication is used for status and location updates. The backend also maintains persistent emergency information in the database and coordinates communication between the matching and fallback components.

Key functions:

- Emergency request processing
- User authentication and authorization
- Ambulance assignment management
- Emergency-state management
- Real-time event communication
- Database interaction
- Notification and status distribution

#### **4.2.5 MODULE 5: INTELLIGENT MATCHER AND ROUTE EVALUATION**

The Intelligent Matcher is responsible for identifying a suitable ambulance for an emergency request. Initially, unavailable or previously attempted ambulances are excluded from consideration, and H3-based spatial filtering is used to identify relevant candidates. Dijkstra's

algorithm is then applied to the available road-network graph to determine route costs between candidate ambulances and the emergency location. The resulting candidates can be ranked using the configured selection factors, including route travel time, physical distance, traffic-related cost, and ambulance availability. The highest-ranked eligible candidate is then forwarded to the dispatch process.

### Scoring Formula

For each eligible ambulance  $A_i$ :

$$\text{Score}(A_i) = w_t T_i + w_d D_i + w_c C_i + w_a P_i$$

Where:

- $T_i$  = normalized route travel time
- $D_i$  = normalized route distance
- $C_i$  = normalized traffic cost
- $P_i$  = availability/eligibility factor
- $w_t, w_d, w_c, w_a$  = corresponding weights

Key functions:

- H3-based spatial candidate filtering
- Available-ambulance identification
- Dijkstra-based route computation
- Candidate scoring and ranking
- Exclusion of previously attempted ambulances
- Selection of the next suitable ambulance

## **4.2.6 MODULE 6: CASCADING FALLBACK AND LIVE TELEMETRY TRACKING**

The Cascading Fallback and Live Telemetry Tracking Module provides fault tolerance and continuous visibility throughout the emergency-response process. When an assigned ambulance rejects the request or does not respond within the configured response period, the failed attempt is recorded and the ambulance is excluded from the candidate set for that request. The matching process is then repeated to identify another eligible ambulance. Once an assignment is accepted, ambulance location and status information are periodically transmitted to the backend and distributed to the relevant applications through the real-time communication layer. This allows the emergency request to continue without requiring the bystander to submit a new request.

Key functions:

- Assignment timeout handling
- Rejection handling
- Failed-ambulance exclusion
- Automatic reassignment
- Assignment-attempt recording
- Live ambulance location updates
- Emergency status synchronization
- Estimated time of arrival updates

## **CHAPTER 5**

### **IMPLEMENTATION AND RESULT DISCUSSION**

#### **5.1 IMPLEMENTATION**

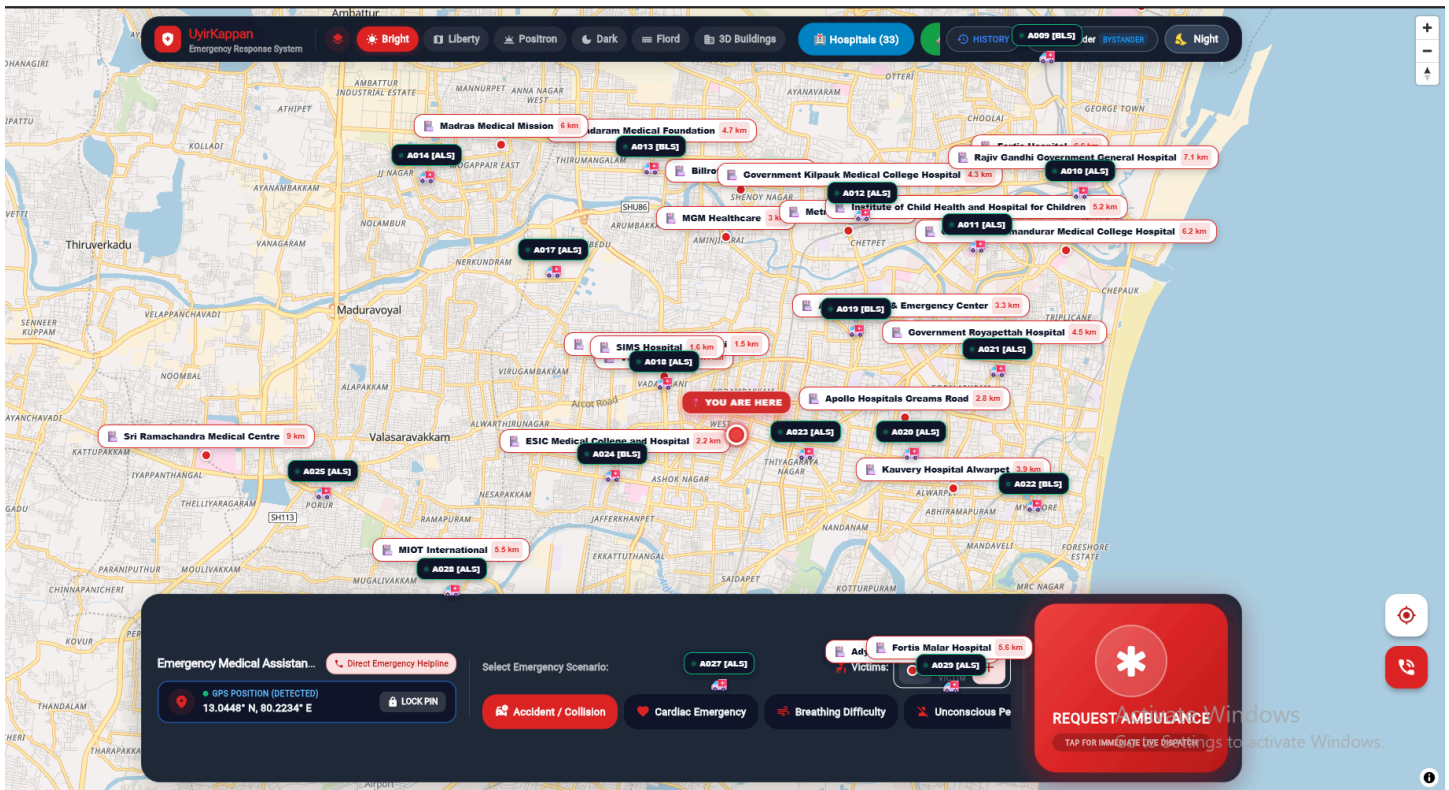
##### **5.1.1 IMPLEMENTATION OF THE EMERGENCY RESPONSE WORKFLOW**

The UyirKappan prototype implements an integrated emergency-response workflow connecting the Bystander Application, Driver/Ambulance Application, Hospital Dashboard, and centralized backend. The workflow begins when a bystander submits an emergency request containing the emergency type, victim information, and geographical location. The request is processed by the backend and forwarded to the Intelligent Matcher for ambulance selection.

The matching process first identifies eligible ambulances using geographical filtering and availability information. H3-based spatial filtering is used to reduce the candidate search area, followed by route evaluation using Dijkstra's algorithm. The candidate ambulances are evaluated using route travel time, distance, traffic-related cost, and availability. The selected ambulance is then presented to the driver through the Driver Application.

Once the driver accepts the assignment, the emergency state is updated and propagated to the connected applications. If the driver rejects the request or fails to respond within the configured response period, the failed attempt is recorded and the fallback mechanism initiates another matching cycle. This allows the system to continue the emergency request without requiring the bystander to submit a new request.

The implemented workflow therefore demonstrates the complete sequence of emergency request creation, ambulance matching, assignment, driver response, fallback handling, and emergency-state synchronization.



**Figure. 5.1. Bystander Application: Emergency Request Interface**

### 5.1.2 IMPLEMENTATION OF REAL-TIME TRACKING AND ETA

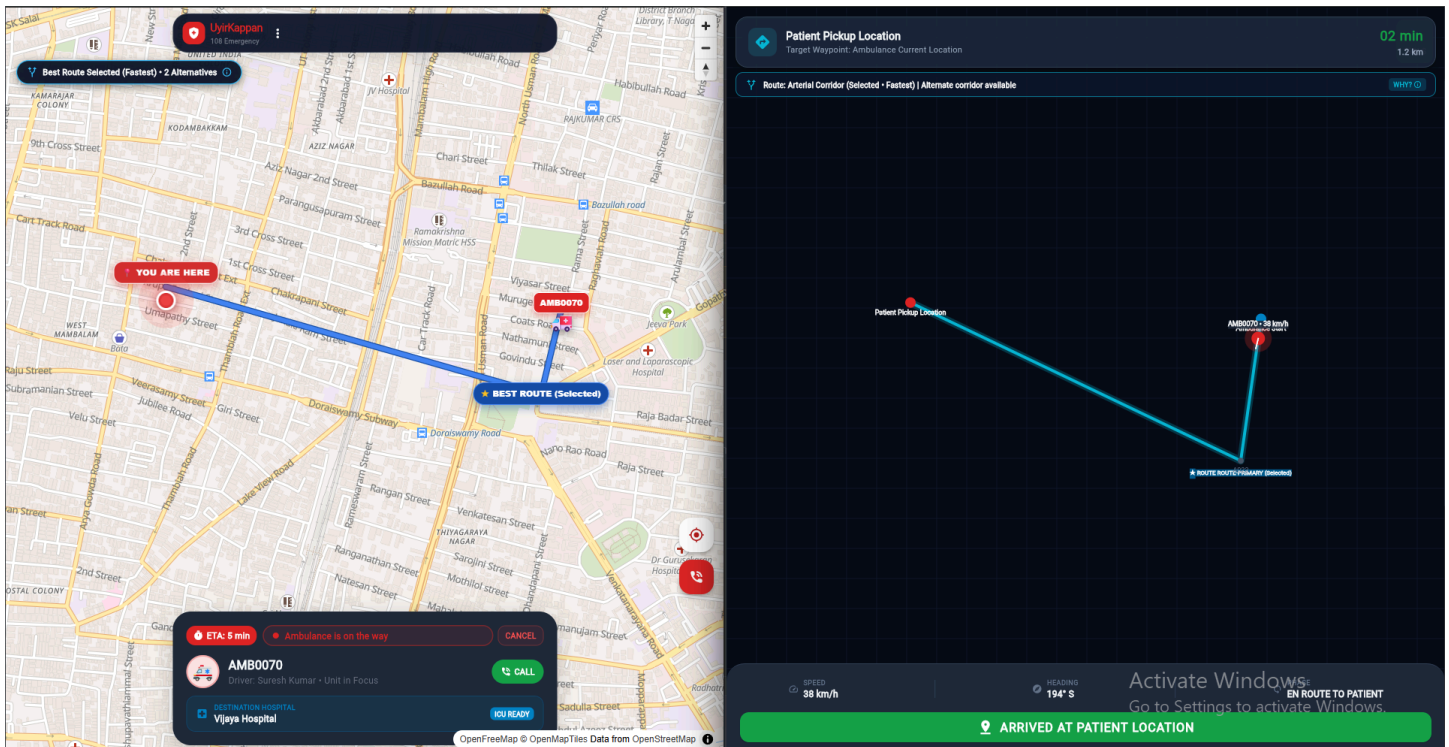
The real-time tracking component was implemented to maintain continuous visibility of the assigned ambulance during an active emergency. After accepting an assignment, the Driver Application periodically transmits the ambulance's current location and status information to the backend. The backend processes these updates and distributes them to the Bystander Application and Hospital Dashboard through the real-time communication layer.

The Bystander Application displays the ambulance location, selected route, emergency status, and estimated time of arrival. The Hospital Dashboard receives the same emergency-state and ambulance information, allowing hospital personnel to monitor the incoming case and prepare the required resources.

The ETA component uses route and travel information to provide an estimated arrival time that is updated as the ambulance progresses along the selected route. This enables the system to

maintain a continuously updated view of the emergency rather than relying only on the initial dispatch estimate.

The implementation demonstrates synchronization of emergency status, ambulance location, and ETA across multiple connected clients. The architecture therefore provides a common real-time view of an emergency throughout the response lifecycle.



**Figure. 5.2. Bystander Application: Live Tracking with ETA and Route**

### 5.1.3 IMPLEMENTATION OF CASCADING FALLBACK AND HOSPITAL COORDINATION

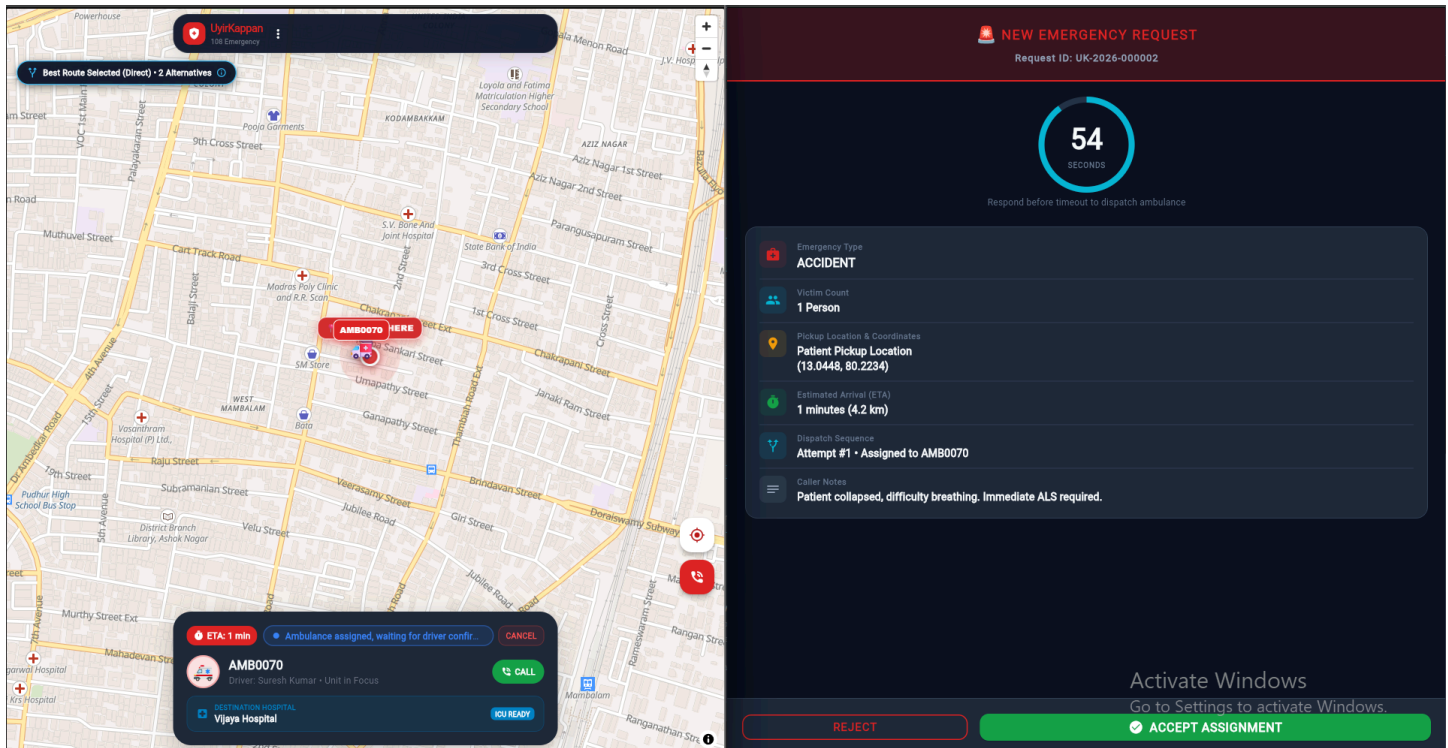
The cascading fallback mechanism was implemented to handle situations in which an ambulance rejects an emergency assignment or fails to respond within the configured response window. Each unsuccessful assignment attempt is recorded by the backend, and the corresponding ambulance is excluded from subsequent matching attempts for the same emergency request. The Intelligent Matcher is then invoked again using the remaining eligible

ambulances.

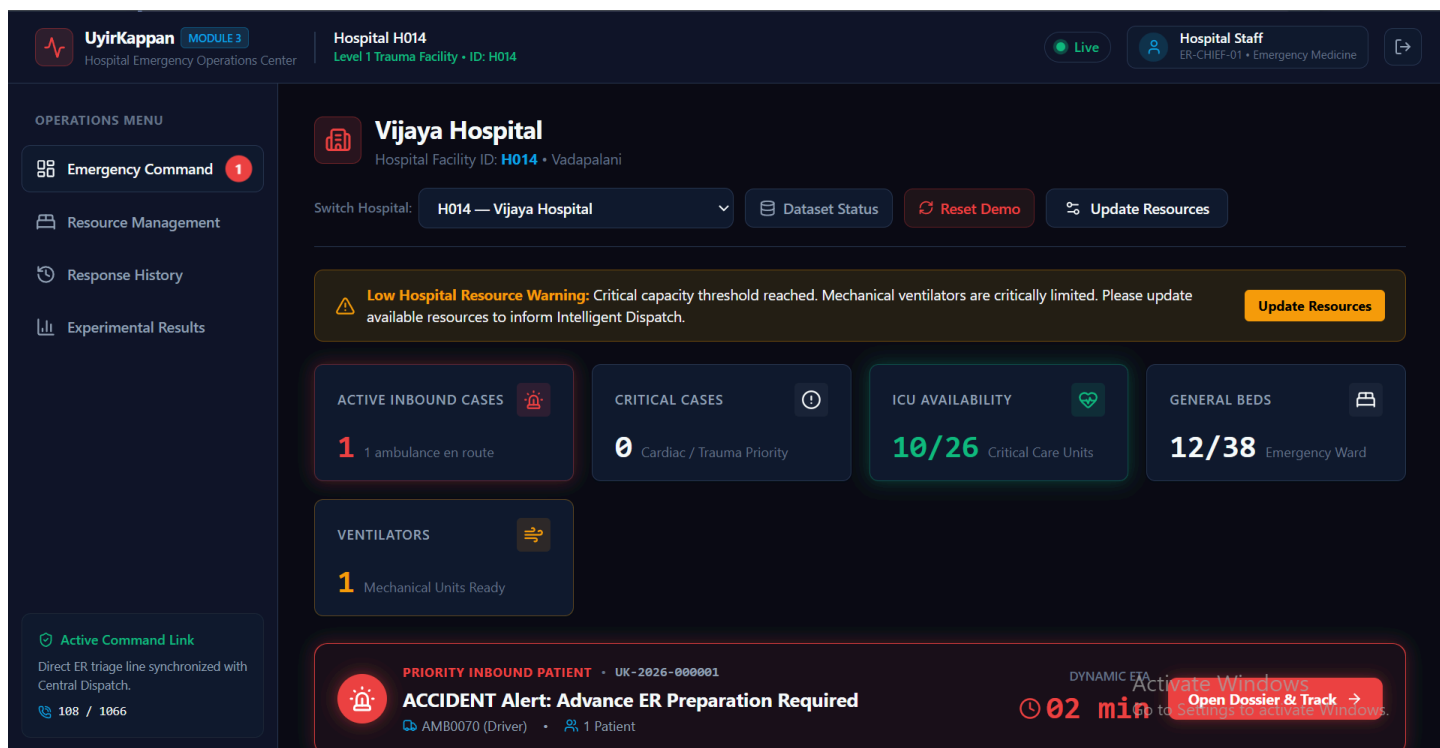
When a new ambulance is successfully assigned, the updated assignment information is propagated to the relevant applications. This process allows an emergency request to continue through multiple assignment attempts without requiring manual re-submission by the bystander.

The Hospital Dashboard provides visibility into incoming emergency cases, ambulance information, estimated arrival time, and emergency status. This enables the receiving hospital to monitor an approaching ambulance and prepare for the patient's arrival. The dashboard also maintains response information that can be used for post-response monitoring and analysis.

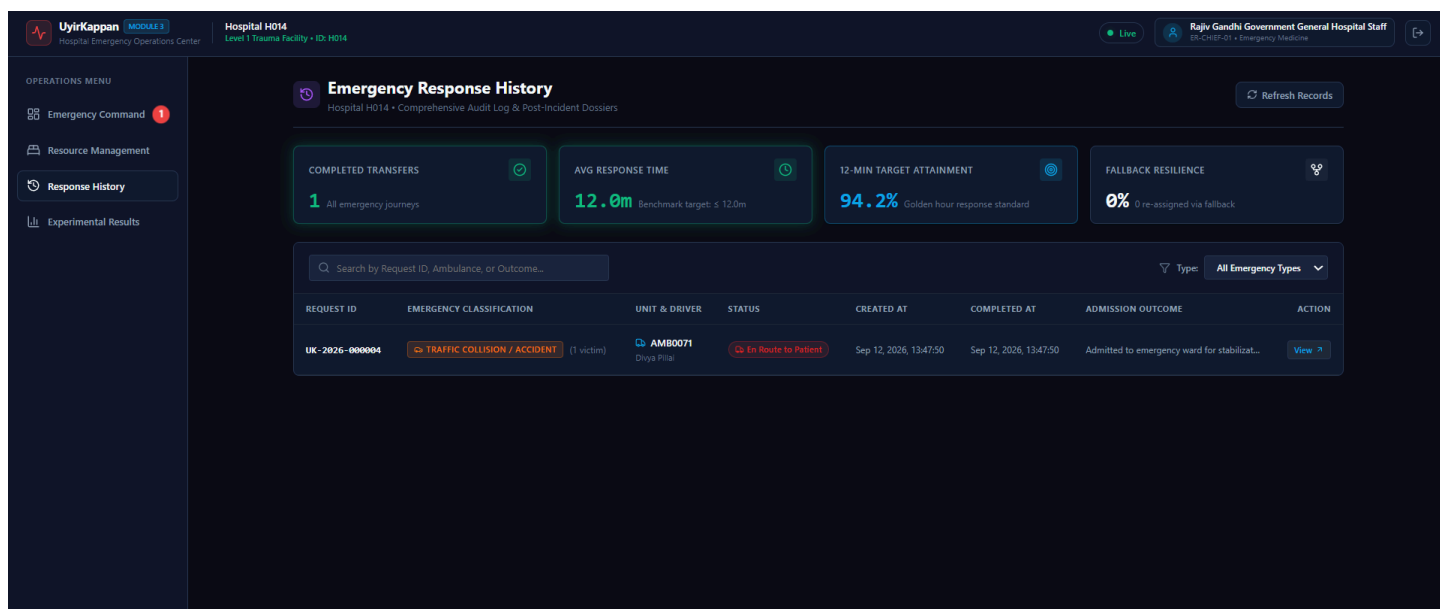
The implementation therefore combines automatic recovery from failed assignments with hospital-side visibility, extending the system beyond ambulance dispatch to provide coordination across the complete emergency-response workflow.



**Figure. 5.3. Driver Application: New Emergency Assignment with Response Timer**



**Figure. 5.4. Hospital Dashboard: Emergency Command and Inbound Patient Alert**

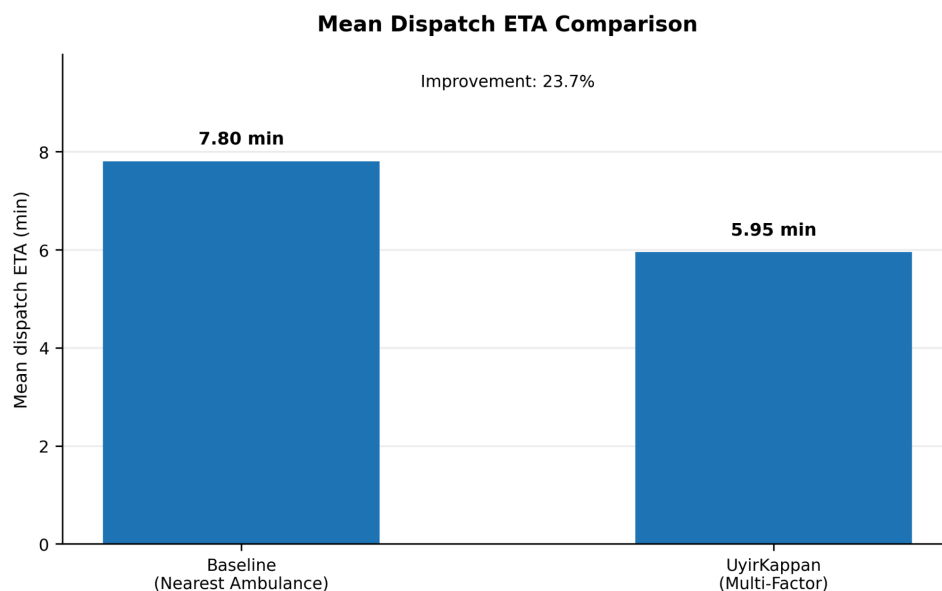


**Figure. 5.5. Hospital Dashboard: Response History and Metrics**

## 5.2 RESULTS AND DISCUSSION

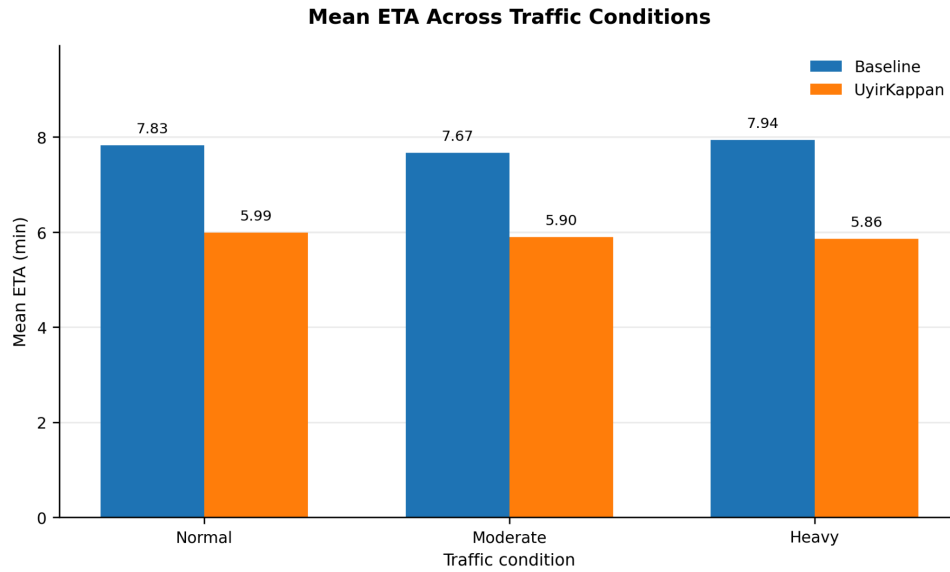
The implemented UyirKappan prototype demonstrates that ambulance dispatch, route evaluation, fallback handling, live tracking, and hospital coordination can be integrated into a single emergency-response workflow. Unlike a nearest-ambulance approach that primarily considers geographical distance, the proposed matching process considers route travel time, distance, traffic-related cost, and ambulance availability.

In the simulation-based evaluation, the mean dispatch ETA obtained using the nearest-ambulance baseline was 7.80 min, while the multi-factor UyirKappan approach produced a mean dispatch ETA of 5.95 min. This corresponds to a 23.7% reduction in mean dispatch ETA. These values represent the results of the implemented simulation/evaluation and should not be presented as measured performance of a live city-wide ambulance service.



**Figure. 5.6. Mean Dispatch ETA Comparison**

The evaluation across different traffic conditions also shows a reduction in mean ETA. Under normal traffic conditions, the mean ETA decreased from 7.83 min to 5.99 min. Under moderate traffic conditions, it decreased from 7.67 min to 5.90 min, while under heavy traffic conditions it decreased from 7.94 min to 5.86 min.



**Figure. 5.7. Mean ETA Comparison under Different Traffic Conditions**

The results indicate that considering route-related factors in ambulance selection can produce different dispatch decisions from selecting an ambulance solely according to geographical proximity. The live tracking implementation further provides continuous ambulance location and ETA information to the connected applications, while the cascading fallback mechanism allows failed assignments to be recovered automatically.

An important aspect of the proposed system is that its contribution is primarily system-level integration rather than the introduction of an entirely new routing algorithm. The prototype combines geospatial filtering, route-based ambulance evaluation, automatic reassignment, real-time synchronization, and hospital coordination within one workflow. This addresses the fragmentation identified in the literature review, where related capabilities are frequently considered separately.

## **CHAPTER 6**

### **CONCLUSION AND FUTURE WORK**

#### **6.1 CONCLUSION**

UyirKappan presents a unified emergency response platform that connects bystanders, ambulance providers, and hospitals through a coordinated digital workflow. The system integrates emergency request creation, geospatial ambulance matching, route-based evaluation, cascading reassignment, live ambulance tracking, and hospital resource coordination. By maintaining continuous communication between the participating applications, the platform reduces dependence on manual coordination and provides a common view of the emergency-response process. The proposed architecture and methodology provide a foundation for reliable ambulance dispatch and coordinated hospital preparedness while allowing the individual modules to operate as part of a single integrated system. The modular design also supports future enhancement of routing, prediction, and emergency coordination capabilities without changing the overall workflow.

The implemented prototype demonstrates the feasibility of integrating these capabilities within a single emergency-response platform. The system provides a continuous flow from emergency request initiation to ambulance selection, driver response, route evaluation, live tracking, fallback handling, and hospital coordination. The simulation-based evaluation also demonstrates the potential benefit of considering multiple dispatch factors rather than relying only on geographical proximity. The results obtained from the implemented evaluation provide a basis for further experimentation and refinement of the proposed approach under different emergency and traffic conditions.

Overall, UyirKappan establishes a system-level framework for coordinated emergency ambulance response by bringing together intelligent dispatch, real-time communication, dynamic ETA estimation, cascading fallback, and hospital-side visibility. Rather than depending on a single algorithm or isolated application, the proposed platform combines these components

into an integrated workflow that can be progressively enhanced and evaluated. The current implementation therefore serves as a foundation for further research, larger-scale simulation, and eventual integration with real-world emergency-response infrastructure.

## **6.2 FUTURE WORK**

Future development of UyirKappan can focus on improving routing accuracy, prediction, and emergency coordination. Traffic-aware routing can be enhanced by incorporating real-time traffic conditions and more detailed road-network data, while machine-learning techniques can be explored for predicting ambulance availability and estimated time of arrival. The platform can also be extended to support in-transit patient vital-sign telemetry and improved hospital selection based on emergency requirements and resource availability.

Further enhancements can include traffic-signal priority for emergency vehicles, integration with external emergency-service networks, improved scalability for larger ambulance fleets, and advanced analytics for evaluating dispatch performance. Large-scale testing under diverse traffic conditions and simulated emergency scenarios can also be conducted to evaluate the reliability, scalability, and operational effectiveness of the platform.

In addition, future implementations can investigate more advanced ambulance allocation and redeployment strategies based on historical demand patterns, emergency frequency, and geographical distribution. The system can be evaluated using larger datasets and more diverse simulation scenarios to examine its behaviour under varying ambulance availability, traffic conditions, and simultaneous emergency requests. Such evaluation can help identify performance limitations and guide improvements to the matching, routing, fallback, and synchronization components.

The platform can also be extended gradually from a simulation-oriented prototype toward controlled pilot environments involving selected ambulances and hospitals. Such future work

would allow the communication workflow, tracking mechanisms, ETA estimation, and hospital coordination functions to be evaluated with real operational data while maintaining appropriate safety and validation procedures. These extensions would help establish the practical applicability of the proposed architecture while preserving its modular design and allowing individual components to be improved independently.

## REFERENCES

- [1] W. Rafaqat, S. M. A. Abidi, J. Lee, A. A. Javed, and A. Mian, "EMCON: A comprehensive emergency response system for low- and middle-income countries," *Disaster Medicine and Public Health Preparedness*, vol. 19, Art. no. e324, 2025.
- [2] C. M. Smith, R. Lall, R. T. Fothergill, R. Spaight, and G. D. Perkins, "The effect of the GoodSAM volunteer first-responder app on survival to hospital discharge following out-of-hospital cardiac arrest," *European Heart Journal—Acute Cardiovascular Care*, vol. 11, no. 1, pp. 20–31, 2022.
- [3] Y. Yang, S. W. Umboh, and M. Ramezani, "Freelance drivers with a decline choice: Dispatch menus in on-demand mobility services for assortment optimization," *Transportation Research Part B: Methodological*, vol. 190, Art. no. 103082, 2024.
- [4] X. Fu, V. Krzhizhanovskaya, A. Yakovlev, and S. Kovalchuk, "Modelling hospital strategies in city-scale ambulance dispatching," *arXiv:2201.01846*, 2022.
- [5] A. Olivier, M. Adams, S. Mohammadi, A. Smyth, K. Thomson, T. Kepler, and M. Dadlani, "Data analytics for improved closest hospital suggestion for EMS operations in New York City," *Sustainable Cities and Society*, vol. 86, Art. no. 104104, 2022.
- [6] Y.-Y. Xu, S.-J. Weng, P.-W. Huang, L.-M. Wang, C.-H. Chen, Y.-T. Tsai, and M.-C. Tsai, "The emergency medical service dispatch recommendation system using simulation based on bed availability," *BMC Health Services Research*, vol. 24, Art. no. 1513, 2024.
- [7] J. Zaki, S. M. R. Islam, N. S. Alghamdi, M. Abdullah-Al-Wadud, and K.-S. Kwak, "Introducing cloud-assisted micro-service-based software development framework for healthcare systems," *IEEE Access*, vol. 10, pp. 33332–33348, 2022.
- [8] H. Ankarboina, J. Kumari, A. K. Singh, and A. Bhardwaj, "RACER: Real-time adaptive congestion-aware emergency routing in urban vehicular networks," *IEEE Open Journal of the Communications Society*, vol. 7, pp. 8157–8170, 2026.
- [9] Y. Xu and Y. Li, "A real-time urban traffic congestion prediction framework based on dynamic risk field and multi-source data fusion," *IEEE Access*, vol. 13, pp. 172471–172495, 2025.
- [10] H. Nozari, A. Szmelter-Jarosz, and H. R. Irani, "Designing an ambulance routing optimization model using the combination of machine learning and genetic algorithm in conditions of uncertainty," *Systems and Soft Computing*, vol. 7, Art. no. 200276, 2025.
- [11] M. A. R. Abdeen, M. H. Ahmed, H. Seliem, T. R. Sheltami, T. M. Alghamdi, and M. El-Nainay, "A novel smart ambulance system—Algorithm design, modeling, and performance analysis," *IEEE Access*, vol. 10, pp. 42656–42672, 2022.

- [12] V. Schmid, "Solving the dynamic ambulance relocation and dispatching problem using approximate dynamic programming," *European Journal of Operational Research*, vol. 219, no. 3, pp. 611–621, 2012.
- [13] R. Tluli, A. Badawy, S. Salem, M. Hardan, S. Chauhan, and G. Alinier, "A dynamic redeployment system for critical care paramedic units in Qatar utilizing deep reinforcement learning," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 7, pp. 93–109, 2025.
- [14] K. F. Chu, A. Y. S. Lam, K. H. Tsoi, Z. Huang, and B. P. Y. Loo, "Deep Encoder Cross Network for estimated time of arrival," *IEEE Access*, vol. 11, pp. 76095–76107, 2023.
- [15] M. A. Abid, F. Lorig, J. Holmgren, and J. Petersson, "Ambulance travel time estimation using spatiotemporal data," *Procedia Computer Science*, vol. 238, pp. 265–272, 2024.
- [16] A. Chatterjee, M. W. Gerdes, P. Khatiwada, and A. Prinz, "SFTSDH: Applying Spring Security Framework with TSD-based OAuth2 to protect microservice architecture APIs," *IEEE Access*, vol. 10, pp. 41914–41934, 2022.
- [17] J. García-González, J. Fernández-Andrés, N. Aliane, and J. Sánchez-Soriano, "Big Data and I2X communication infrastructure for traffic optimization and accident prevention on automated roads," *IEEE Access*, vol. 13, pp. 133497–133509, 2025.
- [18] E. Dritsas and M. Trigka, "Database systems in the Big Data era: Architectures, performance, and open challenges," *IEEE Access*, vol. 13, pp. 95068–95084, 2025.
- [19] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo, and E. Andrade, "Benchmarking consistency levels of cloud-distributed NoSQL databases using YCSB," *IEEE Access*, vol. 13, pp. 63428–63438, 2025.
- [20] R. Tluli, A. Badawy, S. Salem, M. Barhamgi, and A. Mohamed, "A survey of machine learning innovations in ambulance services: Allocation, routing, and demand estimation," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 5, pp. 842–872, 2024.